

\$ `terminal --developer-reference`

Linux / macOS Terminal Developer Cheat Sheet

Industry-style practical command reference for software development — per-command and per-flag definitions, syntax, examples and representative output.

```
terminal - bash / zsh

user@machine:~$ pwd
/home/user/project

user@machine:~$ git status
On branch main
```

SCOPE

Core developer command set: navigation, files, text processing, search, permissions, processes, shell environment, networking, archives, package management, Git, SSH, Node.js, npm/npx, VS Code, Cursor, Docker, Chrome and VLC. There is no finite list of every command because installed software can add commands indefinitely.

PORTABILITY: Linux commonly uses GNU utilities while macOS commonly uses BSD utilities. A flag can differ or be unavailable. Verify with `COMMAND --help` or `man COMMAND`.

01 - Navigation

pwd

\$ pwd

DEFINITION
Prints the current working directory.

COMMAND-LEVEL EXAMPLE
pwd

COMMAND-LEVEL OUTPUT
/home/user/project

EVERY FLAG / FORM — definition • syntax • example • output

-L	logical path, preserving symlinks
SYNTAX	<code>pwd -L</code>
EXAMPLE	<code>pwd -L</code>
OUTPUT	<code>/home/user/project</code>
-P	physical path, resolving symlinks
SYNTAX	<code>pwd -P</code>
EXAMPLE	<code>pwd -P</code>
OUTPUT	<code>/real/path/project</code>

USE / NOTES

Use ``pwd -P`` when you specifically need the physical filesystem path.

 cd**\$ cd****DEFINITION**

Changes the shell's current working directory.

COMMAND-LEVEL EXAMPLE

cd /var/log

COMMAND-LEVEL OUTPUT

(no output)

EVERY FLAG / FORM — definition • syntax • example • output

DIR	go to directory
SYNTAX	cd DIR
EXAMPLE	cd ~/projects
OUTPUT	(no output)
..	parent directory
SYNTAX	cd ..
EXAMPLE	cd ..
OUTPUT	(no output)
-	previous directory
SYNTAX	cd -
EXAMPLE	cd -
OUTPUT	/home/user
~	home directory
SYNTAX	cd ~
EXAMPLE	cd ~
OUTPUT	(no output)

USE / NOTES``cd`` is a shell builtin, not a separate executable. ls**\$ ls****DEFINITION**

Lists directory contents.

COMMAND-LEVEL EXAMPLE

ls -lah

COMMAND-LEVEL OUTPUT

```
total 32
drwxr-xr-x 5 user user 4.0K .
drwxr-xr-x 10 user user 4.0K ..
-rw-r--r-- 1 user user 1.2K app.js
```

EVERY FLAG / FORM — definition • syntax • example • output

-l	long listing
SYNTAX	ls -l PATH
EXAMPLE	ls -l app.js
OUTPUT	-rw-r--r-- 1 user user 1234 Sep 22 19:00 app.js
-a	include hidden entries
SYNTAX	ls -a
EXAMPLE	ls -a
OUTPUT	git app.js

-A	include almost all hidden entries, excluding . and ..
SYNTAX	ls -A
EXAMPLE	ls -A
OUTPUT	.git app.js
-h	human-readable sizes
SYNTAX	ls -lh
EXAMPLE	ls -lh app.js
OUTPUT	-rw-r--r-- ... 1.2K ... app.js
-R	recursive listing
SYNTAX	ls -R PATH
EXAMPLE	ls -R src
OUTPUT	src: app.js utils/
-t	sort by modification time
SYNTAX	ls -lt
EXAMPLE	ls -lt
OUTPUT	new.js old.js
-r	reverse sort/order
SYNTAX	ls -lr
EXAMPLE	ls -lr
OUTPUT	z.txt a.txt
-S	sort by size
SYNTAX	ls -lS
EXAMPLE	ls -lS
OUTPUT	large.bin app.js
-l	one entry per line
SYNTAX	ls -l
EXAMPLE	ls -l
OUTPUT	a.txt b.txt
-d	list directory itself
SYNTAX	ls -d DIR
EXAMPLE	ls -d src
OUTPUT	src
-F	append type indicators
SYNTAX	ls -F
EXAMPLE	ls -F
OUTPUT	src/ run.sh* file.txt

USE / NOTES

On macOS, some color/format flags differ from GNU/Linux.

tree

\$ tree

DEFINITION

Displays directories as a tree. Often installed separately.

COMMAND-LEVEL EXAMPLE

```
tree -L 2 .
```

COMMAND-LEVEL OUTPUT

```
project
├─ src
│  └─ app.js
│  └─ utils
└─ package.json
```

EVERY FLAG / FORM — definition • syntax • example • output

-L N	limit tree depth
SYNTAX	tree -L N PATH
EXAMPLE	tree -L 2 project
OUTPUT	project ├─ src │ └─ app.js
-a	show hidden files
SYNTAX	tree -a PATH
EXAMPLE	tree -a .
OUTPUT	. ├─ .git └─ app.js
-d	directories only
SYNTAX	tree -d PATH
EXAMPLE	tree -d src
OUTPUT	src ├─ components └─ utils

USE / NOTES

Optional utility, but useful for understanding project structure.

02 · Files & Directories

mkdir

\$ mkdir

DEFINITION

Creates directories.

COMMAND-LEVEL EXAMPLE

```
mkdir -p src/components
```

COMMAND-LEVEL OUTPUT

```
(no output)
```

EVERY FLAG / FORM — definition • syntax • example • output

-p	create parent directories as needed
SYNTAX	mkdir -p DIR
EXAMPLE	mkdir -p src/components/ui
OUTPUT	(no output)
-m MODE	set permissions on creation
SYNTAX	mkdir -m 755 DIR

EXAMPLE	mkdir -m 755 public
OUTPUT	(no output)
-v	verbose creation
SYNTAX	mkdir -v DIR
EXAMPLE	mkdir -v demo
OUTPUT	mkdir: created directory 'demo'

touch

\$ touch

DEFINITION
Creates an empty file if it does not exist, or updates timestamps.

COMMAND-LEVEL EXAMPLE

```
touch app.js
```

COMMAND-LEVEL OUTPUT

```
(no output)
```

EVERY FLAG / FORM — definition • syntax • example • output

-a	change access time
SYNTAX	touch -a FILE
EXAMPLE	touch -a app.js
OUTPUT	(no output)
-m	change modification time
SYNTAX	touch -m FILE
EXAMPLE	touch -m app.js
OUTPUT	(no output)
-c	do not create missing file
SYNTAX	touch -c FILE
EXAMPLE	touch -c missing.txt
OUTPUT	(no output; file stays absent)
-r FILE	copy timestamps from reference
SYNTAX	touch -r REF FILE
EXAMPLE	touch -r old.js new.js
OUTPUT	(no output)
-t TIME	set timestamp explicitly
SYNTAX	touch -t [[CC]YY]MMDDhhmm[.ss] FILE
EXAMPLE	touch -t 202609221900 app.js
OUTPUT	(no output)

cp

\$ cp

DEFINITION
Copies files and directories.

COMMAND-LEVEL EXAMPLE

```
cp -r src backup
```

COMMAND-LEVEL OUTPUT

```
(no output)
```

EVERY FLAG / FORM — definition • syntax • example • output

-r / -R	copy directories recursively
SYNTAX	cp -r SRC DEST

EXAMPLE	cp -r src backup
OUTPUT	(no output)
-i	prompt before overwrite
SYNTAX	cp -i SRC DEST
EXAMPLE	cp -i app.js backup.js
OUTPUT	overwrite 'backup.js'? y
-n	do not overwrite existing destination
SYNTAX	cp -n SRC DEST
EXAMPLE	cp -n app.js backup.js
OUTPUT	(no output; existing file preserved)
-v	verbose copy
SYNTAX	cp -v SRC DEST
EXAMPLE	cp -v app.js backup.js
OUTPUT	'app.js' -> 'backup.js'
-p	preserve mode/ownership/timestamps where possible
SYNTAX	cp -p SRC DEST
EXAMPLE	cp -p app.js backup.js
OUTPUT	(no output)
-a	archive copy; preserves attributes and recurses
SYNTAX	cp -a SRC DEST
EXAMPLE	cp -a project backup
OUTPUT	(no output)

mv

\$ mv

DEFINITION

Moves or renames files/directories.

COMMAND-LEVEL EXAMPLE

mv old.js new.js

COMMAND-LEVEL OUTPUT

(no output)

EVERY FLAG / FORM — definition • syntax • example • output

-i	prompt before overwrite
SYNTAX	mv -i SRC DEST
EXAMPLE	mv -i old.js new.js
OUTPUT	overwrite 'new.js'? y
-n	do not overwrite destination
SYNTAX	mv -n SRC DEST
EXAMPLE	mv -n old.js new.js
OUTPUT	(no output; destination preserved)
-v	verbose move
SYNTAX	mv -v SRC DEST
EXAMPLE	mv -v old.js new.js
OUTPUT	renamed 'old.js' -> 'new.js'
-f	force replacement where permitted
SYNTAX	mv -f SRC DEST
EXAMPLE	mv -f old.js new.js

OUTPUT	(no output)
--------	-------------

rm

\$ rm

DEFINITION
Removes files and directories.

COMMAND-LEVEL EXAMPLE
rm -r build

COMMAND-LEVEL OUTPUT
(no output)

EVERY FLAG / FORM — definition • syntax • example • output

-i	prompt before each removal
SYNTAX	rm -i FILE
EXAMPLE	rm -i file.txt
OUTPUT	remove 'file.txt'? y
-I	prompt once before many removals (GNU)
SYNTAX	rm -I FILE...
EXAMPLE	rm -I *.log
OUTPUT	remove 12 arguments? y
-f	force; suppress prompts for some cases
SYNTAX	rm -f FILE
EXAMPLE	rm -f missing.txt
OUTPUT	(no output)
-r / -R	remove directories recursively
SYNTAX	rm -r DIR
EXAMPLE	rm -r build
OUTPUT	(no output)
-v	verbose removal
SYNTAX	rm -v FILE
EXAMPLE	rm -v file.txt
OUTPUT	removed 'file.txt'
-d	remove empty directories
SYNTAX	rm -d DIR
EXAMPLE	rm -d empty
OUTPUT	(no output)

USE / NOTES

⚠ **BE CAREFUL:** `rm` permanently removes data; verify paths before using `-r` or `-f`.

rmdir

\$ rmdir

DEFINITION
Removes empty directories.

COMMAND-LEVEL EXAMPLE
rmdir empty

COMMAND-LEVEL OUTPUT
(no output)

EVERY FLAG / FORM — definition • syntax • example • output

-p	remove empty parent directories too
-----------	-------------------------------------

SYNTAX	<code>rmdir -p a/b/c</code>
EXAMPLE	<code>rmdir -p a/b/c</code>
OUTPUT	(no output)
-v	verbose
SYNTAX	<code>rmdir -v DIR</code>
EXAMPLE	<code>rmdir -v empty</code>
OUTPUT	<code>rmdir: removing directory, 'empty'</code>

ln

\$ ln
DEFINITION
Creates hard links or symbolic links.
COMMAND-LEVEL EXAMPLE
`ln -s /opt/app current`
COMMAND-LEVEL OUTPUT
(no output)

EVERY FLAG / FORM — definition • syntax • example • output

-s	create symbolic link
SYNTAX	<code>ln -s TARGET LINK</code>
EXAMPLE	<code>ln -s /var/www/app current</code>
OUTPUT	(no output)
-f	force replacement where permitted
SYNTAX	<code>ln -sf TARGET LINK</code>
EXAMPLE	<code>ln -sf new current</code>
OUTPUT	(no output)
-i	prompt before replacement
SYNTAX	<code>ln -si TARGET LINK</code>
EXAMPLE	<code>ln -si new current</code>
OUTPUT	replace 'current'? y
-v	verbose
SYNTAX	<code>ln -sv TARGET LINK</code>
EXAMPLE	<code>ln -sv new current</code>
OUTPUT	'current' -> 'new'
-n	treat symlink destination as normal file in relevant implementations
SYNTAX	<code>ln -sn TARGET LINK</code>
EXAMPLE	<code>ln -sn new current</code>
OUTPUT	(no output)

find

\$ find
DEFINITION
Searches directory trees by name, type, time, size and other predicates.
COMMAND-LEVEL EXAMPLE
`find . -type f -name '*.js'`
COMMAND-LEVEL OUTPUT
./src/app.js
./src/server.js

EVERY FLAG / FORM — definition • syntax • example • output

-name	match filename pattern
SYNTAX	find PATH -name PATTERN
EXAMPLE	find . -name '*.js'
OUTPUT	./src/app.js
-iname	case-insensitive name match
SYNTAX	find PATH -iname PATTERN
EXAMPLE	find . -iname '*.JPG'
OUTPUT	./img/photo.jpg
-type f / d	files / directories
SYNTAX	find PATH -type f
EXAMPLE	find . -type f
OUTPUT	./src/app.js
-size	match file size
SYNTAX	find PATH -size N
EXAMPLE	find . -size +100M
OUTPUT	./videos/movie.mp4
-mtime	match modification age in days
SYNTAX	find PATH -mtime N
EXAMPLE	find . -mtime -1
OUTPUT	./logs/today.log
-user	match owner
SYNTAX	find PATH -user USER
EXAMPLE	find . -user user
OUTPUT	./project/app.js
-maxdepth N	limit depth; GNU/Linux
SYNTAX	find PATH -maxdepth N ...
EXAMPLE	find . -maxdepth 2 -type f
OUTPUT	./src/app.js
-exec	execute command for each match
SYNTAX	find ... -exec CMD {} \;
EXAMPLE	find . -name '*.log' -exec wc -l {} \;
OUTPUT	42 ./app.log
-print	print matches
SYNTAX	find ... -print
EXAMPLE	find . -name '*.js' -print
OUTPUT	./src/app.js
-delete	delete matches
SYNTAX	find ... -delete
EXAMPLE	find . -name '*.tmp' -delete
OUTPUT	(no output)

USE / NOTES

⚠ **BE CAREFUL:** Review destructive predicates such as ``-delete`` before running them.

03 · Reading & Text Processing

cat

\$ cat

DEFINITION

Concatenates and prints files.

COMMAND-LEVEL EXAMPLE

```
cat package.json
```

COMMAND-LEVEL OUTPUT

```
{ "name": "demo" }
```

EVERY FLAG / FORM — definition • syntax • example • output

-n	number all lines
SYNTAX	cat -n FILE
EXAMPLE	cat -n file.txt
OUTPUT	1 Hello 2 World
-b	number nonblank lines
SYNTAX	cat -b FILE
EXAMPLE	cat -b file.txt
OUTPUT	1 Hello 2 World
-s	squeeze repeated blank lines
SYNTAX	cat -s FILE
EXAMPLE	cat -s file.txt
OUTPUT	Hello World
-E	show line endings
SYNTAX	cat -E FILE
EXAMPLE	cat -E file.txt
OUTPUT	Hello\$ World\$
-T	show tabs
SYNTAX	cat -T FILE
EXAMPLE	cat -T file.txt
OUTPUT	Hello^IWorld
-A	show nonprinting characters
SYNTAX	cat -A FILE
EXAMPLE	cat -A file.txt
OUTPUT	Hello^IWorld\$

less

\$ less

DEFINITION

Interactive pager for large text output.

COMMAND-LEVEL EXAMPLE

```
less app.log
```

COMMAND-LEVEL OUTPUT

```
(interactive pager; press q to quit)
```

EVERY FLAG / FORM — definition • syntax • example • output

-N	show line numbers
SYNTAX	less -N FILE

EXAMPLE	less -N app.log
OUTPUT	1 first log line
-S	chop long lines instead of wrapping
SYNTAX	less -S FILE
EXAMPLE	less -S app.log
OUTPUT	long line stays on one row
-F	quit if content fits one screen
SYNTAX	less -F FILE
EXAMPLE	less -F small.txt
OUTPUT	small file displayed; exits
-X	do not clear screen on exit
SYNTAX	less -X FILE
EXAMPLE	less -X README.md
OUTPUT	README remains visible

head

\$ head

DEFINITION
Prints the beginning of files.

COMMAND-LEVEL EXAMPLE

```
head -n 5 app.log
```

COMMAND-LEVEL OUTPUT

```
line 1
line 2
line 3
line 4
line 5
```

EVERY FLAG / FORM — definition • syntax • example • output

-n N	show first N lines
SYNTAX	head -n N FILE
EXAMPLE	head -n 5 file.txt
OUTPUT	lines 1-5
-c N	show first N bytes
SYNTAX	head -c N FILE
EXAMPLE	head -c 10 file.txt
OUTPUT	first 10 bytes

tail

\$ tail

DEFINITION
Prints the end of files and can follow appended data.

COMMAND-LEVEL EXAMPLE

```
tail -f app.log
```

COMMAND-LEVEL OUTPUT

```
2026-09-22 INFO server started
2026-09-22 GET /api/profile 200
```

EVERY FLAG / FORM — definition • syntax • example • output

-n N	show last N lines
------	-------------------

SYNTAX	<code>tail -n N FILE</code>
EXAMPLE	<code>tail -n 20 app.log</code>
OUTPUT	last 20 lines
-c N	show last N bytes
SYNTAX	<code>tail -c N FILE</code>
EXAMPLE	<code>tail -c 20 app.log</code>
OUTPUT	last 20 bytes
-f	follow appended data
SYNTAX	<code>tail -f FILE</code>
EXAMPLE	<code>tail -f app.log</code>
OUTPUT	new log lines appear live
-F	follow and retry after rotation (GNU)
SYNTAX	<code>tail -F FILE</code>
EXAMPLE	<code>tail -F app.log</code>
OUTPUT	waiting for new file / new lines
-q	quiet file headers
SYNTAX	<code>tail -q a.log b.log</code>
EXAMPLE	<code>tail -q a.log b.log</code>
OUTPUT	last lines without headers
-v	always show file headers
SYNTAX	<code>tail -v app.log</code>
EXAMPLE	<code>tail -v app.log</code>
OUTPUT	<code>==> app.log <==</code>

WC

\$ WC

DEFINITION
Counts lines, words, bytes or characters.

COMMAND-LEVEL EXAMPLE

```
wc -l app.log
```

COMMAND-LEVEL OUTPUT

```
250 app.log
```

EVERY FLAG / FORM — definition • syntax • example • output

-l	count lines
SYNTAX	<code>wc -l FILE</code>
EXAMPLE	<code>wc -l app.log</code>
OUTPUT	250 app.log
-w	count words
SYNTAX	<code>wc -w FILE</code>
EXAMPLE	<code>wc -w README.md</code>
OUTPUT	820 README.md
-c	count bytes
SYNTAX	<code>wc -c FILE</code>
EXAMPLE	<code>wc -c file.txt</code>
OUTPUT	1234 file.txt
-m	count characters
SYNTAX	<code>wc -m FILE</code>

EXAMPLE	wc -m file.txt
OUTPUT	1234 file.txt
-L	maximum line length; support varies
SYNTAX	wc -L FILE
EXAMPLE	wc -L file.txt
OUTPUT	120 file.txt

sort

\$ sort

DEFINITION
Sorts lines.

COMMAND-LEVEL EXAMPLE

```
sort -n numbers.txt
```

COMMAND-LEVEL OUTPUT

```
1
2
10
```

EVERY FLAG / FORM — definition • syntax • example • output

-n	numeric sort
SYNTAX	sort -n FILE
EXAMPLE	sort -n numbers.txt
OUTPUT	1 2 10
-r	reverse order
SYNTAX	sort -r FILE
EXAMPLE	sort -r names.txt
OUTPUT	Zoe Bob Alice
-k N	sort by key/field
SYNTAX	sort -k N FILE
EXAMPLE	sort -k 2 users.txt
OUTPUT	rows sorted by field 2
-t CHAR	field delimiter
SYNTAX	sort -t, -k2 FILE
EXAMPLE	sort -t, -k2 users.csv
OUTPUT	rows sorted by column 2
-f	case-insensitive sort
SYNTAX	sort -f FILE
EXAMPLE	sort -f names.txt
OUTPUT	alice Bob Charlie
-u	unique output
SYNTAX	sort -u FILE
EXAMPLE	sort -u names.txt
OUTPUT	Alice Bob
-h	human-readable numeric sort (GNU)

SYNTAX	sort -h FILE
EXAMPLE	sort -h sizes.txt
OUTPUT	1K 2M 1G

uniq

\$ uniq

DEFINITION
Filters adjacent duplicate lines; usually combined with sort.

COMMAND-LEVEL EXAMPLE

```
sort names.txt | uniq -c
```

COMMAND-LEVEL OUTPUT

```
2 Alice
1 Bob
```

EVERY FLAG / FORM — definition • syntax • example • output

-c	count occurrences
SYNTAX	uniq -c FILE
EXAMPLE	sort names.txt uniq -c
OUTPUT	2 Alice 1 Bob
-d	show only duplicated lines
SYNTAX	uniq -d FILE
EXAMPLE	sort names.txt uniq -d
OUTPUT	Alice
-u	show only unique lines
SYNTAX	uniq -u FILE
EXAMPLE	sort names.txt uniq -u
OUTPUT	Bob
-i	ignore case while comparing
SYNTAX	uniq -i FILE
EXAMPLE	sort -f names.txt uniq -i
OUTPUT	Alice

cut

\$ cut

DEFINITION
Extracts columns/characters from each line.

COMMAND-LEVEL EXAMPLE

```
cut -d, -f1 users.csv
```

COMMAND-LEVEL OUTPUT

```
Shivanshu
```

EVERY FLAG / FORM — definition • syntax • example • output

-d CHAR	field delimiter
SYNTAX	cut -dCHAR -fLIST FILE
EXAMPLE	cut -d, -f1 users.csv
OUTPUT	Shivanshu
-f LIST	select fields
SYNTAX	cut -fLIST FILE
EXAMPLE	cut -d, -f1,3 users.csv

OUTPUT	Shivanshu,India
-c LIST	select character positions
SYNTAX	cut -cLIST FILE
EXAMPLE	cut -c1-5 file.txt
OUTPUT	Hello
-b LIST	select byte positions
SYNTAX	cut -bLIST FILE
EXAMPLE	cut -b1-5 file.txt
OUTPUT	Hello
-s	suppress lines without delimiter
SYNTAX	cut -d, -s -f1 file.csv
EXAMPLE	cut -d, -s -f1 file.csv
OUTPUT	only delimited rows

tr

\$ tr

DEFINITION

Translates, squeezes or deletes characters.

COMMAND-LEVEL EXAMPLE

echo 'hello' | tr 'a-z' 'A-Z'

COMMAND-LEVEL OUTPUT

HELLO

EVERY FLAG / FORM — definition • syntax • example • output

-d	delete characters
SYNTAX	tr -d SET
EXAMPLE	echo 'abc' tr -d b
OUTPUT	ac
-s	squeeze repeats
SYNTAX	tr -s SET
EXAMPLE	echo 'a b' tr -s ' '
OUTPUT	a b
-c / -C	use complement of character set
SYNTAX	tr -cd SET
EXAMPLE	printf 'a1b2' tr -cd '0-9'
OUTPUT	12

04 · Search & Filtering

grep

\$ grep

DEFINITION

Searches text using patterns.

COMMAND-LEVEL EXAMPLE

grep -rin 'todo' src/

COMMAND-LEVEL OUTPUT

src/app.js:12:// TODO: validate input

EVERY FLAG / FORM — definition • syntax • example • output

-i	ignore case
-----------	-------------

SYNTAX	grep -i PATTERN FILE
EXAMPLE	grep -i error app.log
OUTPUT	Error: connection failed
-n	show line numbers
SYNTAX	grep -n PATTERN FILE
EXAMPLE	grep -n error app.log
OUTPUT	42:error: connection failed
-r / -R	search recursively
SYNTAX	grep -r PATTERN DIR
EXAMPLE	grep -ri todo src/
OUTPUT	src/app.js:12:// TODO
-v	invert match
SYNTAX	grep -v PATTERN FILE
EXAMPLE	grep -v debug app.log
OUTPUT	info: server started
-w	whole-word match
SYNTAX	grep -w cat file.txt
EXAMPLE	grep -w cat words.txt
OUTPUT	cat
-x	whole-line match
SYNTAX	grep -x OK status.txt
EXAMPLE	grep -x OK status.txt
OUTPUT	OK
-c	count matching lines
SYNTAX	grep -c error app.log
EXAMPLE	grep -c error app.log
OUTPUT	7
-l	print filenames with matches
SYNTAX	grep -l error *.log
EXAMPLE	grep -l error *.log
OUTPUT	app.log
-L	print filenames without matches
SYNTAX	grep -L error *.log
EXAMPLE	grep -L error *.log
OUTPUT	access.log
-E	extended regular expressions
SYNTAX	grep -E 'foo bar' file.txt
EXAMPLE	grep -E 'foo bar' file.txt
OUTPUT	foo
-F	fixed-string matching
SYNTAX	grep -F 'a.b' file.txt
EXAMPLE	grep -F 'a.b' file.txt
OUTPUT	a.b
-o	print only matching text
SYNTAX	grep -o error app.log
EXAMPLE	grep -o error app.log

OUTPUT	error
-A N	N lines after match
SYNTAX	grep -A 2 error app.log
EXAMPLE	grep -A 2 error app.log
OUTPUT	error next next
-B N	N lines before match
SYNTAX	grep -B 2 error app.log
EXAMPLE	grep -B 2 error app.log
OUTPUT	prev prev error
-C N	N lines around match
SYNTAX	grep -C 2 error app.log
EXAMPLE	grep -C 2 error app.log
OUTPUT	prev error next

05 - Permissions & Ownership

chmod

\$ **chmod**

DEFINITION

Changes file mode bits.

COMMAND-LEVEL EXAMPLE

chmod +x script.sh

COMMAND-LEVEL OUTPUT

(no output)

EVERY FLAG / FORM — definition • syntax • example • output

-R	recursive permission change
SYNTAX	chmod -R MODE PATH
EXAMPLE	chmod -R 755 scripts
OUTPUT	(no output)
-v	verbose
SYNTAX	chmod -v MODE FILE
EXAMPLE	chmod -v 755 app.sh
OUTPUT	mode of 'app.sh' changed from 644 to 755
-c	report only changed files
SYNTAX	chmod -c MODE FILE
EXAMPLE	chmod -c 755 app.sh
OUTPUT	mode of 'app.sh' changed from 644 to 755
-f	suppress most errors
SYNTAX	chmod -f MODE FILE
EXAMPLE	chmod -f 755 missing
OUTPUT	(no output)

USE / NOTES

Prefer targeted permissions; avoid recursively granting broad write/execute access.

chown

\$ chown

DEFINITION

Changes file owner/group; commonly requires sudo.

COMMAND-LEVEL EXAMPLE

```
sudo chown -R user:group app
```

COMMAND-LEVEL OUTPUT

(no output)

EVERY FLAG / FORM — definition • syntax • example • output

-R	recursive ownership change
SYNTAX	chown -R USER:GROUP PATH
EXAMPLE	sudo chown -R user:group app
OUTPUT	(no output)
-v	verbose
SYNTAX	chown -v USER:GROUP FILE
EXAMPLE	sudo chown -v user:group app
OUTPUT	changed ownership of 'app'
-c	report only changed files
SYNTAX	chown -c USER:GROUP FILE
EXAMPLE	sudo chown -c user:group app
OUTPUT	changed ownership of 'app'
-f	suppress most errors
SYNTAX	chown -f USER:GROUP FILE
EXAMPLE	sudo chown -f user:group missing
OUTPUT	(no output)

USE / NOTES

⚠ **BE CAREFUL:** Recursive ownership changes can break system or application access.

06 - Processes & Job Control

ps

\$ ps

DEFINITION

Reports running processes.

COMMAND-LEVEL EXAMPLE

```
ps aux | grep node
```

COMMAND-LEVEL OUTPUT

```
user 1234 2.1 1.2 ... node server.js
```

EVERY FLAG / FORM — definition • syntax • example • output

-e	select all processes
SYNTAX	ps -e
EXAMPLE	ps -e
OUTPUT	PID TTY TIME CMD
-f	full-format listing
SYNTAX	ps -ef
EXAMPLE	ps -ef
OUTPUT	UID PID PPID C STIME TTY TIME CMD
-u USER	processes for user

SYNTAX	ps -u USER
EXAMPLE	ps -u user
OUTPUT	PID TTY TIME CMD
-p PID	select PID
SYNTAX	ps -p PID
EXAMPLE	ps -p 1234
OUTPUT	PID TTY TIME CMD 1234 pts/0 00:00:00 node
aux	BSD-style all users/detail
SYNTAX	ps aux
EXAMPLE	ps aux
OUTPUT	USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND

top

\$ top

DEFINITION
 Interactive live process/resource monitor.

COMMAND-LEVEL EXAMPLE

```
top
```

COMMAND-LEVEL OUTPUT

```
live CPU/memory/process view
```

EVERY FLAG / FORM — definition • syntax • example • output

-o FIELD	sort/select field; syntax differs by OS
SYNTAX	top -o FIELD
EXAMPLE	top -o cpu
OUTPUT	live process list
-p PID	monitor selected PID where supported
SYNTAX	top -p PID
EXAMPLE	top -p 1234
OUTPUT	node process shown

USE / NOTES

macOS and Linux `top` options differ substantially; check `man top`.

kill

\$ kill

DEFINITION
 Sends a signal to a process.

COMMAND-LEVEL EXAMPLE

```
kill -TERM 1234
```

COMMAND-LEVEL OUTPUT

```
(no output)
```

EVERY FLAG / FORM — definition • syntax • example • output

-l	list signal names
SYNTAX	kill -l
EXAMPLE	kill -l
OUTPUT	1) SIGHUP 2) SIGINT 9) SIGKILL 15) SIGTERM
-s SIGNAL	select signal
SYNTAX	kill -s SIGNAL PID
EXAMPLE	kill -s TERM 1234

OUTPUT	(no output)
-TERM / -15	request graceful termination
SYNTAX	kill -TERM PID
EXAMPLE	kill -TERM 1234
OUTPUT	(no output)
-KILL / -9	force immediate termination
SYNTAX	kill -KILL PID
EXAMPLE	kill -KILL 1234
OUTPUT	(no output)
-HUP / -1	send hangup; often used for reload
SYNTAX	kill -HUP PID
EXAMPLE	kill -HUP 1234
OUTPUT	(no output)

USE / NOTES

⚠ **BE CAREFUL:** Confirm the PID. SIGKILL cannot be caught or cleaned up by the target process.

● ● ● jobs

\$ jobs

DEFINITION

Lists shell jobs started in the current shell.

COMMAND-LEVEL EXAMPLE

```
jobs -l
```

COMMAND-LEVEL OUTPUT

```
[1]+ 1234 Running npm run dev &
```

EVERY FLAG / FORM — definition • syntax • example • output

-l	include process IDs
SYNTAX	jobs -l
EXAMPLE	jobs -l
OUTPUT	[1]+ 1234 Running ./server.sh &
-p	show PIDs only
SYNTAX	jobs -p
EXAMPLE	jobs -p
OUTPUT	1234
-r	running jobs
SYNTAX	jobs -r
EXAMPLE	jobs -r
OUTPUT	[1]+ Running ./server.sh &
-s	stopped jobs
SYNTAX	jobs -s
EXAMPLE	jobs -s
OUTPUT	[1]+ Stopped ./server.sh

fg

\$ fg

DEFINITION

Brings a stopped/background job to the foreground.

COMMAND-LEVEL EXAMPLE

fg %1

COMMAND-LEVEL OUTPUT

npm run dev

EVERY FLAG / FORM — definition • syntax • example • output

JOB	job specification such as %1
SYNTAX	fg %JOB
EXAMPLE	fg %1
OUTPUT	npm run dev

USE / NOTES

Shell builtin.

bg

\$ bg

DEFINITION

Continues a stopped job in the background.

COMMAND-LEVEL EXAMPLE

bg %1

COMMAND-LEVEL OUTPUT

[1]+ npm run dev &

EVERY FLAG / FORM — definition • syntax • example • output

JOB	job specification
SYNTAX	bg %JOB
EXAMPLE	bg %1
OUTPUT	[1]+ npm run dev &

USE / NOTES

Shell builtin.

lsof

\$ lsof

DEFINITION

Lists open files; useful for finding which process owns a port.

COMMAND-LEVEL EXAMPLE

lsof -i :3000

COMMAND-LEVEL OUTPUT

node 1234 user ... TCP *:3000 (LISTEN)

EVERY FLAG / FORM — definition • syntax • example • output

-i :PORT	network files for a port
SYNTAX	lsof -i :PORT
EXAMPLE	lsof -i :3000
OUTPUT	node 1234 user 20u IPv4 ... TCP *:3000 (LISTEN)
-i	network files
SYNTAX	lsof -i
EXAMPLE	lsof -i
OUTPUT	COMMAND PID USER ...

-p PID	files for a process
SYNTAX	lsof -p PID
EXAMPLE	lsof -p 1234
OUTPUT	node 1234 ...
-n	do not resolve hostnames
SYNTAX	lsof -n -i :3000
EXAMPLE	lsof -n -i :3000
OUTPUT	127.0.0.1:3000

07 · Shell Environment

env

\$ env

DEFINITION

Runs a command with a modified environment or prints environment.

COMMAND-LEVEL EXAMPLE

env | grep PATH

COMMAND-LEVEL OUTPUT

PATH=/usr/local/bin:/usr/bin:/bin

EVERY FLAG / FORM — definition • syntax • example • output

-i	ignore inherited environment
SYNTAX	env -i COMMAND
EXAMPLE	env -i env
OUTPUT	(empty environment)
-u NAME	unset variable
SYNTAX	env -u NAME COMMAND
EXAMPLE	env -u PATH env
OUTPUT	PATH absent
-0	NUL-terminate output; GNU
SYNTAX	env -0
EXAMPLE	env -0
OUTPUT	KEY=value\0...

export

\$ export

DEFINITION

Marks shell variables for export to child processes.

COMMAND-LEVEL EXAMPLE

export PORT=3000 && node server.js

COMMAND-LEVEL OUTPUT

server listening on 3000

EVERY FLAG / FORM — definition • syntax • example • output

-p	print exported variables
SYNTAX	export -p
EXAMPLE	export -p
OUTPUT	declare -x PATH=...

USE / NOTES

Shell builtin.

source

\$ source

DEFINITION

Executes a file in the current shell process.

COMMAND-LEVEL EXAMPLE

```
source ~/.bashrc
```

COMMAND-LEVEL OUTPUT

(no output)

EVERY FLAG / FORM — definition • syntax • example • output

FILE	read and execute commands from file
SYNTAX	source FILE
EXAMPLE	source ~/.bashrc
OUTPUT	(no output)

USE / NOTES

Often used after changing shell configuration.

08 - Command Discovery & Help

which

\$ which

DEFINITION

Shows which executable would be found in PATH.

COMMAND-LEVEL EXAMPLE

```
which node
```

COMMAND-LEVEL OUTPUT

```
/usr/bin/node
```

EVERY FLAG / FORM — definition • syntax • example • output

-a	show all matching executables; availability varies
SYNTAX	which -a COMMAND
EXAMPLE	which -a node
OUTPUT	/usr/local/bin/node /usr/bin/node

whereis

\$ whereis

DEFINITION

Locates binaries, manuals and source files.

COMMAND-LEVEL EXAMPLE

```
whereis node
```

COMMAND-LEVEL OUTPUT

```
node: /usr/bin/node /usr/share/man/man1/node.1
```

EVERY FLAG / FORM — definition • syntax • example • output

-b	search for binary
SYNTAX	whereis -b COMMAND
EXAMPLE	whereis -b node
OUTPUT	node: /usr/bin/node
-m	search manuals
SYNTAX	whereis -m COMMAND
EXAMPLE	whereis -m ls
OUTPUT	ls: /usr/share/man/man1/ls.1

-s	search source
SYNTAX	whereis -s COMMAND
EXAMPLE	whereis -s bash
OUTPUT	bash: /usr/src/bash

type

\$ type
DEFINITION
Tells how the shell resolves a command name.
COMMAND-LEVEL EXAMPLE

```
type -a python
```

COMMAND-LEVEL OUTPUT

```
python is /usr/bin/python
```

EVERY FLAG / FORM — definition • syntax • example • output

-a	show all matching command types/locations in Bash
SYNTAX	type -a NAME
EXAMPLE	type -a node
OUTPUT	node is /usr/local/bin/node node is /usr/bin/node
-t	show type only in Bash
SYNTAX	type -t NAME
EXAMPLE	type -t cd
OUTPUT	builtin

09 · Networking & HTTP / APIs

curl

\$ curl
DEFINITION
Transfers data using URLs; essential for HTTP/API testing.
COMMAND-LEVEL EXAMPLE

```
curl -X GET 'https://api.example.com/profile' -H 'Accept: application/json'
```

COMMAND-LEVEL OUTPUT

```
{"id":123,"name":"Shivanshu"}
```

EVERY FLAG / FORM — definition • syntax • example • output

-I	headers only
SYNTAX	curl -I URL
EXAMPLE	curl -I https://example.com
OUTPUT	HTTP/2 200
-i	include headers in output
SYNTAX	curl -i URL
EXAMPLE	curl -i https://example.com
OUTPUT	HTTP/2 200 + body
-L	follow redirects
SYNTAX	curl -L URL
EXAMPLE	curl -L http://example.com
OUTPUT	final response
-s	silent progress/errors
SYNTAX	curl -s URL

EXAMPLE	curl -s https://example.com
OUTPUT	HTML body
-S	show errors when using -s
SYNTAX	curl -sS URL
EXAMPLE	curl -sS https://bad.example
OUTPUT	curl: (6) Could not resolve host
-o FILE	write output to file
SYNTAX	curl -o FILE URL
EXAMPLE	curl -o page.html https://example.com
OUTPUT	(no body printed)
-O	use remote filename
SYNTAX	curl -O URL
EXAMPLE	curl -O https://example.com/file.zip
OUTPUT	% Total ...
-X METHOD	set HTTP method
SYNTAX	curl -X POST URL
EXAMPLE	curl -X POST https://api.example.com
OUTPUT	API response
-H HEADER	add request header
SYNTAX	curl -H 'Accept: application/json' URL
EXAMPLE	curl -H 'Accept: application/json' https://api.example.com
OUTPUT	JSON response
-d DATA	send request body
SYNTAX	curl -d '{"x":1}' URL
EXAMPLE	curl -d '{"x":1}' https://api.example.com
OUTPUT	API response
-u USER:PASS	HTTP basic auth credentials
SYNTAX	curl -u USER:PASS URL
EXAMPLE	curl -u user:pass https://example.com
OUTPUT	authenticated response
-k	skip TLS certificate verification
SYNTAX	curl -k URL
EXAMPLE	curl -k https://selfsigned.example
OUTPUT	response body
-v	verbose protocol/connection details
SYNTAX	curl -v URL
EXAMPLE	curl -v https://example.com
OUTPUT	> GET / ... < HTTP/2 200

USE / NOTES

⚠ **BE CAREFUL:** `-k` disables TLS certificate verification and should not be used to bypass real certificate errors in production.

• • • wget

\$ wget

DEFINITION

Command-line downloader, common on Linux.

COMMAND-LEVEL EXAMPLE

```
wget https://example.com/file.zip
```

COMMAND-LEVEL OUTPUT

```
Saving to: 'file.zip'
```

EVERY FLAG / FORM — definition • syntax • example • output

-O FILE	save as specified file
SYNTAX	wget -O FILE URL
EXAMPLE	wget -O page.html https://example.com
OUTPUT	Saving to: 'page.html'
-c	continue partial download
SYNTAX	wget -c URL
EXAMPLE	wget -c big.iso
OUTPUT	Resuming at ...
-q	quiet
SYNTAX	wget -q URL
EXAMPLE	wget -q https://example.com/file.zip
OUTPUT	(no output)
-r	recursive download
SYNTAX	wget -r URL
EXAMPLE	wget -r https://example.com/docs/
OUTPUT	... saved
-i FILE	read URLs from file
SYNTAX	wget -i urls.txt
EXAMPLE	wget -i urls.txt
OUTPUT	Downloading ...
--limit-rate=RATE	limit download rate
SYNTAX	wget --limit-rate=500k URL
EXAMPLE	wget --limit-rate=500k URL
OUTPUT	... 500 KB/s

USE / NOTES

macOS often uses curl by default; wget may need installation.

• • • ping

\$ ping

DEFINITION

Tests IP reachability and measures round-trip time.

COMMAND-LEVEL EXAMPLE

```
ping -c 4 1.1.1.1
```

COMMAND-LEVEL OUTPUT

```
64 bytes from 1.1.1.1: icmp_seq=0 ttl=... time=20 ms
```

EVERY FLAG / FORM — definition • syntax • example • output

-c N	send N packets
SYNTAX	ping -c N HOST

EXAMPLE	ping -c 4 example.com
OUTPUT	4 packets transmitted, 4 received
-W TIME	timeout; units differ by OS
SYNTAX	ping -W 2 HOST
EXAMPLE	ping -W 2 example.com
OUTPUT	64 bytes from ... time=20 ms
-i SECONDS	packet interval
SYNTAX	ping -i 2 HOST
EXAMPLE	ping -i 2 example.com
OUTPUT	64 bytes from ...

SS

\$ ss

DEFINITION
Shows socket statistics on Linux; macOS commonly uses `lsof -i` or `netstat`.

COMMAND-LEVEL EXAMPLE

```
ss -lntp
```

COMMAND-LEVEL OUTPUT

```
LISTEN 0 511 127.0.0.1:3000 0.0.0.0:* users: (('node',pid=1234))
```

EVERY FLAG / FORM — definition • syntax • example • output

-t	TCP sockets
SYNTAX	ss -t
EXAMPLE	ss -t
OUTPUT	State Recv-Q Send-Q Local Address:Port
-u	UDP sockets
SYNTAX	ss -u
EXAMPLE	ss -u
OUTPUT	UNCONN ...
-l	listening sockets
SYNTAX	ss -l
EXAMPLE	ss -l
OUTPUT	LISTEN ...
-n	numeric addresses/ports
SYNTAX	ss -n
EXAMPLE	ss -n
OUTPUT	127.0.0.1:3000
-p	show process information
SYNTAX	ss -p
EXAMPLE	ss -p
OUTPUT	users: (('node',pid=1234))
-a	all sockets
SYNTAX	ss -a
EXAMPLE	ss -a
OUTPUT	LISTEN ... ESTAB ...
-4 / -6	IPv4 / IPv6
SYNTAX	ss -4
EXAMPLE	ss -4

OUTPUT	tcp ... 0.0.0.0:80
--------	--------------------

USE / NOTES
Linux-focused.

```
dig
```

\$ dig

DEFINITION
Queries DNS records.

COMMAND-LEVEL EXAMPLE

```
dig example.com A
```

COMMAND-LEVEL OUTPUT

```
;; ANSWER SECTION:  
example.com. 300 IN A 93.184.216.34
```

EVERY FLAG / FORM — definition • syntax • example • output

+short	compact answer
SYNTAX	dig +short NAME TYPE
EXAMPLE	dig +short example.com A
OUTPUT	93.184.216.34
+trace	trace DNS delegation
SYNTAX	dig +trace NAME
EXAMPLE	dig +trace example.com
OUTPUT	root → TLD → authoritative
@SERVER	query specific DNS server
SYNTAX	dig @SERVER NAME TYPE
EXAMPLE	dig @1.1.1.1 example.com A
OUTPUT	ANSWER SECTION ...

```
ip
```

\$ ip

DEFINITION
Shows and configures Linux interfaces, addresses, routes and neighbors.

COMMAND-LEVEL EXAMPLE

```
ip addr show
```

COMMAND-LEVEL OUTPUT

```
2: eth0: <UP> ... inet 192.168.1.10/24
```

EVERY FLAG / FORM — definition • syntax • example • output

addr	addresses
SYNTAX	ip addr
EXAMPLE	ip addr
OUTPUT	2: eth0 ... inet 192.168.1.10/24
link	interfaces
SYNTAX	ip link
EXAMPLE	ip link
OUTPUT	2: eth0: <BROADCAST,MULTICAST,UP>
route	routing table
SYNTAX	ip route
EXAMPLE	ip route
OUTPUT	default via 192.168.1.1 dev eth0

neigh	neighbor/ARP cache
SYNTAX	ip neigh
EXAMPLE	ip neigh
OUTPUT	192.168.1.1 dev eth0 lladdr aa:bb:cc ...
rule	routing policy rules
SYNTAX	ip rule
EXAMPLE	ip rule
OUTPUT	0: from all lookup local
-br	brief output
SYNTAX	ip -br addr
EXAMPLE	ip -br addr
OUTPUT	eth0 UP 192.168.1.10/24
-4 / -6	IPv4 / IPv6 only
SYNTAX	ip -4 addr
EXAMPLE	ip -4 addr
OUTPUT	inet 192.168.1.10/24

USE / NOTES

Linux-focused; macOS uses different networking commands.

10 · Archives & Compression

tar

\$ tar

DEFINITION

Creates, lists and extracts tar archives; compression is selected by flags.

COMMAND-LEVEL EXAMPLE

```
tar -czf project.tar.gz project/
```

COMMAND-LEVEL OUTPUT

(no output)

EVERY FLAG / FORM — definition • syntax • example • output

-c	create archive
SYNTAX	tar -cf ARCHIVE FILES
EXAMPLE	tar -cf backup.tar project/
OUTPUT	(no output)
-x	extract archive
SYNTAX	tar -xf ARCHIVE
EXAMPLE	tar -xf backup.tar
OUTPUT	(no output)
-t	list archive
SYNTAX	tar -tf ARCHIVE
EXAMPLE	tar -tf backup.tar
OUTPUT	project/ project/app.js
-f	archive filename
SYNTAX	tar -cf ARCHIVE FILES
EXAMPLE	tar -cf backup.tar project/
OUTPUT	(no output)
-v	verbose files

SYNTAX	tar -cvf backup.tar project/
EXAMPLE	tar -cvf backup.tar project/
OUTPUT	project/ project/app.js
-z	gzip compression
SYNTAX	tar -czf ARCHIVE.tar.gz DIR
EXAMPLE	tar -czf backup.tar.gz project/
OUTPUT	(no output)
-j	bzip2 compression
SYNTAX	tar -cjf ARCHIVE.tar.bz2 DIR
EXAMPLE	tar -cjf backup.tar.bz2 project/
OUTPUT	(no output)
-J	xz compression
SYNTAX	tar -cJf ARCHIVE.tar.xz DIR
EXAMPLE	tar -cJf backup.tar.xz project/
OUTPUT	(no output)
-C DIR	change directory before operation
SYNTAX	tar -C DIR -cf ARCHIVE FILES
EXAMPLE	tar -C /tmp -cf backup.tar project
OUTPUT	(no output)

zip

\$ zip

DEFINITION

Creates ZIP archives.

COMMAND-LEVEL EXAMPLE

zip -r project.zip project/

COMMAND-LEVEL OUTPUT

adding: project/

EVERY FLAG / FORM — definition • syntax • example • output

-r	recursive directory inclusion
SYNTAX	zip -r ARCHIVE DIR
EXAMPLE	zip -r project.zip project/
OUTPUT	adding: project/
-q	quiet
SYNTAX	zip -q ARCHIVE FILE
EXAMPLE	zip -q project.zip app.js
OUTPUT	(no output)
-9	maximum compression
SYNTAX	zip -9 ARCHIVE FILE
EXAMPLE	zip -9 project.zip app.js
OUTPUT	adding: app.js

unzip

\$ **unzip**

DEFINITION

Extracts and inspects ZIP archives.

COMMAND-LEVEL EXAMPLE

unzip project.zip

COMMAND-LEVEL OUTPUT

Archive: project.zip
extracting: project/app.js

EVERY FLAG / FORM — definition • syntax • example • output

-l	list contents
SYNTAX	unzip -l ARCHIVE
EXAMPLE	unzip -l project.zip
OUTPUT	Length Name
-o	overwrite without prompting
SYNTAX	unzip -o ARCHIVE
EXAMPLE	unzip -o project.zip
OUTPUT	Archive: project.zip

11 · System & Package Management

sudo

\$ **sudo**

DEFINITION

Runs a command with elevated privileges according to system policy.

COMMAND-LEVEL EXAMPLE

sudo command

COMMAND-LEVEL OUTPUT

(command output)

EVERY FLAG / FORM — definition • syntax • example • output

-u USER	run as another user
SYNTAX	sudo -u USER COMMAND
EXAMPLE	sudo -u postgres psql
OUTPUT	psql prompt
-k	invalidate cached credentials
SYNTAX	sudo -k
EXAMPLE	sudo -k
OUTPUT	(no output)
-v	refresh/validate credentials
SYNTAX	sudo -v
EXAMPLE	sudo -v
OUTPUT	(no output)
-l	list allowed commands
SYNTAX	sudo -l
EXAMPLE	sudo -l
OUTPUT	User may run the following commands...

USE / NOTES

⚠ BE CAREFUL:

`sudo` can modify system-wide files and settings.

apt

\$ apt

DEFINITION

Debian/Ubuntu package manager.

COMMAND-LEVEL EXAMPLE

```
sudo apt update
```

COMMAND-LEVEL OUTPUT

```
Hit:1 http://archive.ubuntu.com/ubuntu ...
```

EVERY FLAG / FORM — definition • syntax • example • output

update	refresh package indexes
SYNTAX	apt update
EXAMPLE	sudo apt update
OUTPUT	Fetches ...
upgrade	upgrade installed packages
SYNTAX	apt upgrade
EXAMPLE	sudo apt upgrade
OUTPUT	The following packages will be upgraded...
install	install package
SYNTAX	apt install PKG
EXAMPLE	sudo apt install build-essential
OUTPUT	Setting up ...
remove	remove package
SYNTAX	apt remove PKG
EXAMPLE	sudo apt remove package
OUTPUT	Removing ...
purge	remove package plus configuration where supported
SYNTAX	apt purge PKG
EXAMPLE	sudo apt purge package
OUTPUT	Purging ...
search	search packages
SYNTAX	apt search TERM
EXAMPLE	apt search nodejs
OUTPUT	Sorting... Full Text Search...
show	show package details
SYNTAX	apt show PKG
EXAMPLE	apt show nodejs
OUTPUT	Package: nodejs

USE / NOTES

Linux/Debian family only.

brew

\$ brew

DEFINITION

Homebrew package manager; common on macOS and available on Linux.

COMMAND-LEVEL EXAMPLE

```
brew install node
```

COMMAND-LEVEL OUTPUT

```
==> Downloading node...
```

EVERY FLAG / FORM — definition • syntax • example • output

install	install formula
SYNTAX	brew install PKG
EXAMPLE	brew install node
OUTPUT	==> Downloading ...
update	update Homebrew metadata
SYNTAX	brew update
EXAMPLE	brew update
OUTPUT	Updated Homebrew...
upgrade	upgrade installed formulae
SYNTAX	brew upgrade
EXAMPLE	brew upgrade
OUTPUT	==> Upgrading ...
uninstall	remove formula
SYNTAX	brew uninstall PKG
EXAMPLE	brew uninstall node
OUTPUT	Uninstalling ...
search	search formulae
SYNTAX	brew search TERM
EXAMPLE	brew search node
OUTPUT	node node@20 ...
info	show package information
SYNTAX	brew info PKG
EXAMPLE	brew info node
OUTPUT	node: stable ...
list	list installed formulae/casks
SYNTAX	brew list
EXAMPLE	brew list
OUTPUT	node wget ...

USE / NOTES

macOS-focused; not a Bash builtin.

npm

\$ npm

DEFINITION

Node Package Manager; manages project dependencies and scripts.

COMMAND-LEVEL EXAMPLE

```
npm install && npm run dev
```

COMMAND-LEVEL OUTPUT

```
added packages
```

```
> app@1.0.0 dev
```

```
> node server.js
```

EVERY FLAG / FORM — definition • syntax • example • output

-v / --version	show npm version
SYNTAX	npm --version
EXAMPLE	npm --version
OUTPUT	10.x.x
init	create package.json
SYNTAX	npm init
EXAMPLE	npm init
OUTPUT	package name: ...
install / i	install dependencies
SYNTAX	npm install
EXAMPLE	npm install
OUTPUT	added 123 packages
install PKG	install a package
SYNTAX	npm install PKG
EXAMPLE	npm install express
OUTPUT	added 65 packages
install -D PKG	install development dependency
SYNTAX	npm install -D PKG
EXAMPLE	npm install -D nodemon
OUTPUT	added ...
uninstall PKG	remove dependency
SYNTAX	npm uninstall PKG
EXAMPLE	npm uninstall express
OUTPUT	removed ...
update	update packages within allowed ranges
SYNTAX	npm update
EXAMPLE	npm update
OUTPUT	changed ...
run SCRIPT	run package.json script
SYNTAX	npm run SCRIPT
EXAMPLE	npm run dev
OUTPUT	> app@1.0.0 dev > node server.js
exec / x	run package binary/package command
SYNTAX	npm exec CMD
EXAMPLE	npm exec eslint .

OUTPUT	...
list	show dependency tree
SYNTAX	npm list
EXAMPLE	npm list --depth=0
OUTPUT	app@1.0.0 └─ express@...
outdated	show outdated dependencies
SYNTAX	npm outdated
EXAMPLE	npm outdated
OUTPUT	Package Current Wanted Latest
audit	check known dependency vulnerabilities
SYNTAX	npm audit
EXAMPLE	npm audit
OUTPUT	found 0 vulnerabilities
cache	inspect/manage npm cache
SYNTAX	npm cache verify
EXAMPLE	npm cache verify
OUTPUT	Content verified...
ci	clean install from lockfile
SYNTAX	npm ci
EXAMPLE	npm ci
OUTPUT	added ... packages

● ● ● npx

\$ npx

DEFINITION
Runs a package binary without requiring a permanent global install.

COMMAND-LEVEL EXAMPLE

```
npx create-vite@latest
```

COMMAND-LEVEL OUTPUT

```
Need to install the following packages...
✓ Project created
```

EVERY FLAG / FORM — definition • syntax • example • output

-y	automatically answer yes to install prompt when supported
SYNTAX	npm -y PKG
EXAMPLE	npm -y create-vite@latest
OUTPUT	Scaffolding project...
--yes	long form of -y
SYNTAX	npm --yes PKG
EXAMPLE	npm --yes cowsay hello
OUTPUT	...
--package PKG	make a package available to the executed command
SYNTAX	npm --package PKG CMD
EXAMPLE	npm --package typescript tsc --version
OUTPUT	Version 5.x.x

corepack

\$ corepack

DEFINITION

Manages package-manager shims for tools such as pnpm and Yarn in Node versions that include Corepack.

COMMAND-LEVEL EXAMPLE

```
corepack enable
```

COMMAND-LEVEL OUTPUT

```
(no output)
```

EVERY FLAG / FORM — definition • syntax • example • output

enable	enable package-manager shims
SYNTAX	corepack enable
EXAMPLE	corepack enable
OUTPUT	(no output)
prepare	prepare a package-manager version in older Corepack workflows
SYNTAX	corepack prepare MANAGER@VERSION --activate
EXAMPLE	corepack prepare pnpm@latest --activate
OUTPUT	Preparing pnpm...

USE / NOTES

Availability depends on Node.js distribution/version.

uname

\$ uname

DEFINITION

Prints system/kernel information.

COMMAND-LEVEL EXAMPLE

```
uname -a
```

COMMAND-LEVEL OUTPUT

```
Linux my-machine 6.8.0 x86_64 GNU/Linux
```

EVERY FLAG / FORM — definition • syntax • example • output

-a	all available information
SYNTAX	uname -a
EXAMPLE	uname -a
OUTPUT	Linux host 6.x ... x86_64
-s	kernel/system name
SYNTAX	uname -s
EXAMPLE	uname -s
OUTPUT	Linux
-r	kernel release
SYNTAX	uname -r
EXAMPLE	uname -r
OUTPUT	6.8.0-xx
-m	machine hardware name
SYNTAX	uname -m
EXAMPLE	uname -m
OUTPUT	x86_64
-n	network node/hostname
SYNTAX	uname -n
EXAMPLE	uname -n

OUTPUT	my-machine
--------	------------

date

\$ date

DEFINITION
Prints current date/time and can format it.

COMMAND-LEVEL EXAMPLE

```
date '+%Y-%m-%d %H:%M:%S'
```

COMMAND-LEVEL OUTPUT

```
2026-09-22 19:00:00
```

EVERY FLAG / FORM — definition • syntax • example • output

-u	use UTC
SYNTAX	date -u
EXAMPLE	date -u
OUTPUT	Tue Sep 22 13:30:00 UTC 2026
+FORMAT	format output
SYNTAX	date +FORMAT
EXAMPLE	date +%Y-%m-%d
OUTPUT	2026-09-22

12 · Git

git

\$ git

DEFINITION
Distributed version-control tool used in virtually every coding workflow.

COMMAND-LEVEL EXAMPLE

```
git status
```

COMMAND-LEVEL OUTPUT

```
On branch main

Changes not staged for commit:
  modified: app.js
```

EVERY FLAG / FORM — definition • syntax • example • output

init	initialize repository
SYNTAX	git init [DIR]
EXAMPLE	git init
OUTPUT	Initialized empty Git repository
clone	clone remote repository
SYNTAX	git clone URL
EXAMPLE	git clone https://example.com/repo.git
OUTPUT	Cloning into 'repo'...
status	show working tree status
SYNTAX	git status
EXAMPLE	git status
OUTPUT	On branch main
add	stage changes
SYNTAX	git add PATH
EXAMPLE	git add app.js

OUTPUT	(no output)
commit	create commit
SYNTAX	git commit -m MESSAGE
EXAMPLE	git commit -m 'Add API'
OUTPUT	[main a1b2c3d] Add API
log	show commit history
SYNTAX	git log --oneline
EXAMPLE	git log --oneline
OUTPUT	a1b2c3d Add API
diff	show changes
SYNTAX	git diff
EXAMPLE	git diff
OUTPUT	diff --git a/app.js b/app.js
branch	manage branches
SYNTAX	git branch
EXAMPLE	git branch feature
OUTPUT	feature
switch	switch branches
SYNTAX	git switch NAME
EXAMPLE	git switch main
OUTPUT	Switched to branch 'main'
checkout	older multi-purpose branch/file command
SYNTAX	git checkout NAME
EXAMPLE	git checkout main
OUTPUT	Switched to branch 'main'
merge	merge histories
SYNTAX	git merge BRANCH
EXAMPLE	git merge feature
OUTPUT	Merge made by the 'ort' strategy.
rebase	replay commits onto a new base
SYNTAX	git rebase BRANCH
EXAMPLE	git rebase main
OUTPUT	Successfully rebased...
fetch	download remote refs without integrating
SYNTAX	git fetch REMOTE
EXAMPLE	git fetch origin
OUTPUT	From https://example.com/repo
pull	fetch and integrate
SYNTAX	git pull
EXAMPLE	git pull origin main
OUTPUT	Already up to date.
push	send commits to remote
SYNTAX	git push
EXAMPLE	git push origin main
OUTPUT	main -> main
remote	manage remotes

SYNTAX	git remote -v
EXAMPLE	git remote -v
OUTPUT	origin https://example.com/repo.git
stash	temporarily store uncommitted work
SYNTAX	git stash
EXAMPLE	git stash
OUTPUT	Saved working directory...
tag	mark a commit
SYNTAX	git tag NAME
EXAMPLE	git tag v1.0.0
OUTPUT	(no output)
restore	restore working-tree files
SYNTAX	git restore PATH
EXAMPLE	git restore app.js
OUTPUT	(no output)

13 · SSH & Remote Development

ssh

\$ ssh

DEFINITION

Secure remote shell and port-forwarding client.

COMMAND-LEVEL EXAMPLE

ssh user@server

COMMAND-LEVEL OUTPUT

user@server:~\$

EVERY FLAG / FORM — definition • syntax • example • output

-p PORT	remote SSH port
SYNTAX	ssh -p PORT USER@HOST
EXAMPLE	ssh -p 2222 user@server
OUTPUT	user@server:~\$
-i FILE	identity/private key
SYNTAX	ssh -i KEY USER@HOST
EXAMPLE	ssh -i ~/.ssh/id_ed25519 user@server
OUTPUT	user@server:~\$
-v / -vvv	verbose connection debugging
SYNTAX	ssh -vvv USER@HOST
EXAMPLE	ssh -vvv user@server
OUTPUT	OpenSSH_... debug1: ...
-L	local port forwarding
SYNTAX	ssh -L LOCAL:DEST:PORT USER@HOST
EXAMPLE	ssh -L 8080:localhost:3000 user@server
OUTPUT	local :8080 forwards to remote :3000
-R	remote port forwarding
SYNTAX	ssh -R REMOTE:DEST:PORT USER@HOST
EXAMPLE	ssh -R 8080:localhost:3000 user@server
OUTPUT	remote :8080 forwards to local :3000

-N	do not run remote command
SYNTAX	ssh -N -L ... USER@HOST
EXAMPLE	ssh -N -L 8080:localhost:3000 user@server
OUTPUT	(tunnel stays open)

USE / NOTES

Keep private keys protected; use key-based authentication where possible.

scp

\$ scp

DEFINITION

Copies files over SSH.

COMMAND-LEVEL EXAMPLE

```
scp app.js user@server:/tmp/
```

COMMAND-LEVEL OUTPUT

```
app.js 100%
```

EVERY FLAG / FORM — definition • syntax • example • output

-r	recursive directory copy
SYNTAX	scp -r SRC USER@HOST:DEST
EXAMPLE	scp -r project user@server:/tmp/
OUTPUT	project/app.js 100%
-P PORT	SSH port
SYNTAX	scp -P PORT SRC DEST
EXAMPLE	scp -P 2222 app.js user@server:/tmp/
OUTPUT	app.js 100%
-i FILE	identity key
SYNTAX	scp -i KEY SRC DEST
EXAMPLE	scp -i ~/.ssh/id_ed25519 app.js user@server:/tmp/
OUTPUT	app.js 100%
-p	preserve times/modes
SYNTAX	scp -p SRC DEST
EXAMPLE	scp -p app.js backup/
OUTPUT	app.js 100%

rsync

\$ rsync

DEFINITION

Efficiently synchronizes directory trees; common for servers and backups.

COMMAND-LEVEL EXAMPLE

```
rsync -av project/ user@server:/srv/project/
```

COMMAND-LEVEL OUTPUT

```
sending incremental file list
app.js
```

EVERY FLAG / FORM — definition • syntax • example • output

-a	archive mode; recursive + preserve attributes
SYNTAX	rsync -a SRC DEST
EXAMPLE	rsync -a project/ backup/
OUTPUT	sending incremental file list
-v	verbose

SYNTAX	<code>rsync -v SRC DEST</code>
EXAMPLE	<code>rsync -av project/ backup/</code>
OUTPUT	<code>app.js</code>
-z	compress transfer
SYNTAX	<code>rsync -z SRC DEST</code>
EXAMPLE	<code>rsync -az project/ user@server:/tmp/project/</code>
OUTPUT	<code>sent ...</code>
-h	human-readable numbers
SYNTAX	<code>rsync -h SRC DEST</code>
EXAMPLE	<code>rsync -ah project/ backup/</code>
OUTPUT	<code>1.20K</code>
--delete	remove destination files absent from source
SYNTAX	<code>rsync --delete SRC DEST</code>
EXAMPLE	<code>rsync -a --delete project/ backup/</code>
OUTPUT	<code>deleting old.js</code>
-n / --dry-run	simulate changes
SYNTAX	<code>rsync -n SRC DEST</code>
EXAMPLE	<code>rsync -an --delete project/ backup/</code>
OUTPUT	<code>deleting old.js</code>

USE / NOTES

⚠ **BE CAREFUL:** `--delete` can remove destination files.

14 · Node.js Runtime

node

\$ node

DEFINITION

Runs JavaScript outside the browser and provides the Node.js runtime.

COMMAND-LEVEL EXAMPLE

```
node server.js
```

COMMAND-LEVEL OUTPUT

```
Server listening on port 3000
```

EVERY FLAG / FORM — definition • syntax • example • output

-v / --version	print Node.js version
SYNTAX	<code>node --version</code>
EXAMPLE	<code>node --version</code>
OUTPUT	<code>v22.x.x</code>
-e CODE	evaluate JavaScript
SYNTAX	<code>node -e CODE</code>
EXAMPLE	<code>node -e "console.log(2+2)"</code>
OUTPUT	<code>4</code>
-p CODE	evaluate and print result
SYNTAX	<code>node -p CODE</code>
EXAMPLE	<code>node -p "process.platform"</code>
OUTPUT	<code>linux</code>
--check FILE	syntax-check a JS file without executing it
SYNTAX	<code>node --check FILE</code>

EXAMPLE	node --check app.js
OUTPUT	(no output if valid)

16 · Coding Editors

code

\$ code

DEFINITION
VS Code command-line launcher, when the `code` CLI is installed.

COMMAND-LEVEL EXAMPLE

```
code .
```

COMMAND-LEVEL OUTPUT

```
VS Code opens the current project
```

EVERY FLAG / FORM — definition • syntax • example • output

-v	print version
SYNTAX	code -v
EXAMPLE	code -v
OUTPUT	1.xx.x
FILE	open file
SYNTAX	code FILE
EXAMPLE	code app.js
OUTPUT	VS Code opens app.js
DIR	open folder/workspace
SYNTAX	code DIR
EXAMPLE	code .
OUTPUT	VS Code opens current project
-n	new window
SYNTAX	code -n DIR
EXAMPLE	code -n .
OUTPUT	new VS Code window
-r	reuse current window
SYNTAX	code -r DIR
EXAMPLE	code -r .
OUTPUT	current window opens project
--diff A B	compare two files
SYNTAX	code --diff A B
EXAMPLE	code --diff old.js new.js
OUTPUT	diff editor opens
--goto FILE:LINE:COLUMN	open at location
SYNTAX	code --goto FILE:LINE:COLUMN
EXAMPLE	code --goto app.js:20:5
OUTPUT	cursor at line 20, column 5

USE / NOTES

The CLI may be installed separately from the editor.

cursor

\$ cursor

DEFINITION

Cursor editor command-line launcher, when its CLI is installed.

COMMAND-LEVEL EXAMPLE

cursor .

COMMAND-LEVEL OUTPUT

Cursor opens the current project

EVERY FLAG / FORM — definition • syntax • example • output

-v	print version where supported
SYNTAX	cursor -v
EXAMPLE	cursor -v
OUTPUT	Cursor version ...
FILE	open file
SYNTAX	cursor FILE
EXAMPLE	cursor app.js
OUTPUT	Cursor opens app.js
DIR	open project folder
SYNTAX	cursor DIR
EXAMPLE	cursor .
OUTPUT	Cursor opens project
-n	new window where supported
SYNTAX	cursor -n DIR
EXAMPLE	cursor -n .
OUTPUT	new Cursor window
--diff A B	compare files where supported
SYNTAX	cursor --diff A B
EXAMPLE	cursor --diff old.js new.js
OUTPUT	diff editor

USE / NOTES

CLI flags can vary by Cursor release; verify with `cursor --help`.

17 · Docker & Containers

docker

\$ docker

DEFINITION

Builds, runs and manages containers, images, networks and volumes.

COMMAND-LEVEL EXAMPLE

docker ps

COMMAND-LEVEL OUTPUT

CONTAINER ID IMAGE COMMAND STATUS PORTS NAMES

EVERY FLAG / FORM — definition • syntax • example • output

-v / --version	show Docker version
SYNTAX	docker --version
EXAMPLE	docker --version
OUTPUT	Docker version 27.x.x
ps	list containers

SYNTAX	<code>docker ps</code>
EXAMPLE	<code>docker ps</code>
OUTPUT	CONTAINER ID IMAGE COMMAND STATUS PORTS NAMES
ps -a	list all containers
SYNTAX	<code>docker ps -a</code>
EXAMPLE	<code>docker ps -a</code>
OUTPUT	... Exited ...
run	create/start a container
SYNTAX	<code>docker run IMAGE</code>
EXAMPLE	<code>docker run -d -p 3000:3000 node:22</code>
OUTPUT	container id
build	build image
SYNTAX	<code>docker build -t NAME .</code>
EXAMPLE	<code>docker build -t myapp .</code>
OUTPUT	Successfully built ...
pull	download image
SYNTAX	<code>docker pull IMAGE</code>
EXAMPLE	<code>docker pull node:22</code>
OUTPUT	Status: Downloaded newer image...
images	list images
SYNTAX	<code>docker images</code>
EXAMPLE	<code>docker images</code>
OUTPUT	REPOSITORY TAG IMAGE ID
logs	show container logs
SYNTAX	<code>docker logs CONTAINER</code>
EXAMPLE	<code>docker logs api</code>
OUTPUT	server listening...
exec	run command inside running container
SYNTAX	<code>docker exec -it CONTAINER CMD</code>
EXAMPLE	<code>docker exec -it api sh</code>
OUTPUT	/app #
stop	stop container
SYNTAX	<code>docker stop CONTAINER</code>
EXAMPLE	<code>docker stop api</code>
OUTPUT	api
start	start stopped container
SYNTAX	<code>docker start CONTAINER</code>
EXAMPLE	<code>docker start api</code>
OUTPUT	api
restart	restart container
SYNTAX	<code>docker restart CONTAINER</code>
EXAMPLE	<code>docker restart api</code>
OUTPUT	api
rm	remove container
SYNTAX	<code>docker rm CONTAINER</code>
EXAMPLE	<code>docker rm api</code>

OUTPUT	api
rmi	remove image
SYNTAX	docker rmi IMAGE
EXAMPLE	docker rmi myapp
OUTPUT	Untagged: myapp
cp	copy files between host/container
SYNTAX	docker cp SRC CONTAINER:DEST
EXAMPLE	docker cp .env api:/app/.env
OUTPUT	(no output)
inspect	low-level object details
SYNTAX	docker inspect OBJECT
EXAMPLE	docker inspect api
OUTPUT	JSON
volume	manage volumes
SYNTAX	docker volume ls
EXAMPLE	docker volume ls
OUTPUT	DRIVER VOLUME NAME
network	manage networks
SYNTAX	docker network ls
EXAMPLE	docker network ls
OUTPUT	NETWORK ID NAME DRIVER
compose	manage multi-container apps
SYNTAX	docker compose up -d
EXAMPLE	docker compose up -d
OUTPUT	Container api Started

docker compose

\$ docker compose

DEFINITION
Runs multi-container applications defined in compose.yaml/compose.yml.

COMMAND-LEVEL EXAMPLE

```
docker compose up -d
```

COMMAND-LEVEL OUTPUT

```
[+] Running 2/2
✓ Container db Started
✓ Container api Started
```

EVERY FLAG / FORM — definition • syntax • example • output

up	create/start services
SYNTAX	docker compose up
EXAMPLE	docker compose up -d
OUTPUT	Container api Started
-d	detached/background mode
SYNTAX	docker compose up -d
EXAMPLE	docker compose up -d
OUTPUT	Containers started
down	stop/remove compose resources
SYNTAX	docker compose down

EXAMPLE	docker compose down
OUTPUT	Container api Removed
build	build services
SYNTAX	docker compose build
EXAMPLE	docker compose build
OUTPUT	Successfully built...
logs	view service logs
SYNTAX	docker compose logs SERVICE
EXAMPLE	docker compose logs api
OUTPUT	api listening on 3000
ps	list compose containers
SYNTAX	docker compose ps
EXAMPLE	docker compose ps
OUTPUT	NAME SERVICE STATUS
exec	execute inside service
SYNTAX	docker compose exec SERVICE CMD
EXAMPLE	docker compose exec api sh
OUTPUT	/app #
restart	restart services
SYNTAX	docker compose restart
EXAMPLE	docker compose restart api
OUTPUT	Restarting api ... done

18 · Installed Developer/Desktop Apps

google-chrome

\$ google-chrome

DEFINITION
Chrome launcher; exact command name/options depend on installation and OS.

COMMAND-LEVEL EXAMPLE

```
google-chrome http://localhost:3000
```

COMMAND-LEVEL OUTPUT

```
Chrome opens localhost:3000
```

EVERY FLAG / FORM — definition • syntax • example • output

--version	print Chrome version
SYNTAX	google-chrome --version
EXAMPLE	google-chrome --version
OUTPUT	Google Chrome 1xx.x...
--new-window URL	open URL in new window
SYNTAX	google-chrome --new-window URL
EXAMPLE	google-chrome --new-window http://localhost:3000
OUTPUT	browser window opens
--incognito URL	open incognito window
SYNTAX	google-chrome --incognito URL
EXAMPLE	google-chrome --incognito http://localhost:3000
OUTPUT	incognito window opens

--headless URL	run without visible UI
SYNTAX	google-chrome --headless URL
EXAMPLE	google-chrome --headless https://example.com
OUTPUT	headless browser output/process

USE / NOTES

On macOS the launcher is commonly `~/Applications/Google Chrome.app/Contents/MacOS/Google Chrome``; Linux may use ``google-chrome`` or ``google-chrome-stable``.

command

\$ vlc

DEFINITION

VLC command-line/media player; useful for testing media and streams.

COMMAND-LEVEL EXAMPLE

```
vlc video.mp4
```

COMMAND-LEVEL OUTPUT

```
VLC opens video.mp4
```

EVERY FLAG / FORM — definition • syntax • example • output

--version	show VLC version
SYNTAX	vlc --version
EXAMPLE	vlc --version
OUTPUT	VLC media player 3.x...
--fullscreen	start fullscreen
SYNTAX	vlc --fullscreen FILE
EXAMPLE	vlc --fullscreen video.mp4
OUTPUT	video opens fullscreen
--play-and-exit	play then exit
SYNTAX	vlc --play-and-exit FILE
EXAMPLE	vlc --play-and-exit video.mp4
OUTPUT	media plays then VLC exits
--intf dummy	no normal GUI interface
SYNTAX	vlc --intf dummy FILE
EXAMPLE	vlc --intf dummy audio.mp3
OUTPUT	VLC console output

USE / NOTES

VLC flags vary by platform/build.

19 · Shell Utilities

command

\$ command

DEFINITION

Runs a command while bypassing some shell function/alias lookup and can inspect resolution.

COMMAND-LEVEL EXAMPLE

```
command -v npm
```

COMMAND-LEVEL OUTPUT

```
/usr/bin/npm
```

EVERY FLAG / FORM — definition • syntax • example • output

-v	show how command is resolved
-----------	------------------------------

SYNTAX	command -v NAME
EXAMPLE	command -v node
OUTPUT	/usr/bin/node
-V	detailed resolution in Bash
SYNTAX	command -V NAME
EXAMPLE	command -V cd
OUTPUT	cd is a shell builtin

echo

\$ echo

DEFINITION
Prints text or expanded variables.

COMMAND-LEVEL EXAMPLE

```
echo $PATH
```

COMMAND-LEVEL OUTPUT

```
/usr/local/bin:/usr/bin:/bin
```

EVERY FLAG / FORM — definition • syntax • example • output

-n	do not append newline
SYNTAX	echo -n TEXT
EXAMPLE	echo -n Hi
OUTPUT	Hi
-e	interpret backslash escapes in supported implementations
SYNTAX	echo -e 'A\nB'
EXAMPLE	echo -e 'A\nB'
OUTPUT	A B

printf

\$ printf

DEFINITION
Prints formatted text predictably; preferred over echo for portable formatting.

COMMAND-LEVEL EXAMPLE

```
printf 'PORT=%s\n' 3000
```

COMMAND-LEVEL OUTPUT

```
PORT=3000
```

EVERY FLAG / FORM — definition • syntax • example • output

FORMAT	format string
SYNTAX	printf FORMAT [ARG]
EXAMPLE	printf 'Hello %s\n' Shivanshu
OUTPUT	Hello Shivanshu
%s	string conversion
SYNTAX	printf '%s\n' VALUE
EXAMPLE	printf '%s\n' hello
OUTPUT	hello
%d	integer conversion
SYNTAX	printf '%d\n' VALUE
EXAMPLE	printf '%d\n' 42
OUTPUT	42

USE / NOTES

Shell builtin in Bash.

 clear

\$ clear

DEFINITION

Clears the terminal display.

COMMAND-LEVEL EXAMPLE

```
clear
```

COMMAND-LEVEL OUTPUT

```
(terminal cleared)
```

EVERY FLAG / FORM — definition • syntax • example • output

-x	clear screen without clearing scrollbar in some terminals
SYNTAX	clear -x
EXAMPLE	clear -x
OUTPUT	(terminal cleared)

 history

\$ history

DEFINITION

Shows command history.

COMMAND-LEVEL EXAMPLE

```
history | tail
```

COMMAND-LEVEL OUTPUT

```
101 git status
```

```
102 npm run dev
```

EVERY FLAG / FORM — definition • syntax • example • output

-c	clear history in Bash
SYNTAX	history -c
EXAMPLE	history -c
OUTPUT	(no output)
-d N	delete history entry N
SYNTAX	history -d N
EXAMPLE	history -d 102
OUTPUT	(no output)
-w	write history to history file
SYNTAX	history -w
EXAMPLE	history -w
OUTPUT	(no output)

20 · Help & Manuals

man

\$ man

DEFINITION

Reads installed manual pages.

COMMAND-LEVEL EXAMPLE

man curl

COMMAND-LEVEL OUTPUT

CURL(1) ... curl - transfer a URL

EVERY FLAG / FORM — definition • syntax • example • output

-k	search manual descriptions
SYNTAX	man -k KEYWORD
EXAMPLE	man -k network
OUTPUT	network_namespaces (7) ...
-f	show one-line description
SYNTAX	man -f COMMAND
EXAMPLE	man -f ls
OUTPUT	ls (1) - list directory contents

apropos

\$ apropos

DEFINITION

Searches manual-page descriptions; equivalent to `man -k` on many systems.

COMMAND-LEVEL EXAMPLE

apropos archive

COMMAND-LEVEL OUTPUT

tar (1) ... zip (1) ...

EVERY FLAG / FORM — definition • syntax • example • output

-a	match all supplied terms on implementations supporting it
SYNTAX	apropos -a TERM1 TERM2
EXAMPLE	apropos -a network socket
OUTPUT	...

Developer Core — Fast Index

Navigation	pwd · cd · ls · mkdir · tree
Files	touch · cp · mv · rm · rmdir · ln
Text	cat · less · head · tail · wc · sort · uniq · cut · tr
Search	grep · find
Permissions	chmod · chown · sudo
Processes	ps · top · kill · jobs · fg · bg
Environment	env · export · source
Networking	curl · ping · ss · lsof · dig · ip
Archives	tar · zip · unzip
Packages	apt · brew
Git	git
Remote	ssh · scp · rsync
Node	node · npm · npx · corepack
Editors	code · cursor
Containers	docker · docker compose
Installed apps	google-chrome · vlc
Help	man · apropos · type · command · which · whereis

● ● ● portability

```
# Linux/GNU: command --help | man command
# macOS/BSD: command --help | man command
# Check the actual installed version before relying on platform-specific flags.
```